



河南大學
HENAN UNIVERSITY

第1章 计算机系统结构的基础知识

- 1.1 [计算机系统结构的基本概念](#)
- 1.2 [计算机系统的设计](#)
- 1.3 [计算机系统的性能评测](#)
- 1.4 [计算机系统结构的发展](#)
- 1.5 [计算机系统结构中并行性的发展](#)

>>> 1.1 计算机系统结构的基本概念

- 第一台通用电子计算机诞生于1946年
- 计算机技术的飞速发展受益于两个方面
 - ✓ 计算机制造技术的发展（如大规模集成电路的发展）；
 - ✓ 计算机系统结构的创新（(CISC→RISC) → 流水线 + Cache → 单核超标量优化 → 多核并行 → 异构加速）；

>>> 1.1 计算机 system 结构的基本概念

➤ 经历了四个发展过程

时 间	原 因	每年的性能增长
1946年起的25年	两种因素都起着主要的作用	25%
20世纪70年代末 ~80年代初	大规模集成电路和微处理器 出现, 以集成电路为代表的制 造技术的发展	约35%
80年代中开始	➤ 计算机系统结构的重大转折: ✓ 从单纯依靠指令级并行 (一个核心) 转向开发线程级并行和数 据级并行 (多个核心) ;	
2003年以来		

- ✓ 大功耗问题;
- ✓ 可以进一步有效地开发的指令级并行性已经很少;
- ✓ 存储器访问速度的提高缓慢。

>>> 1.1 计算机系统结构的基本概念

1.1.1 计算机系统的层次结构

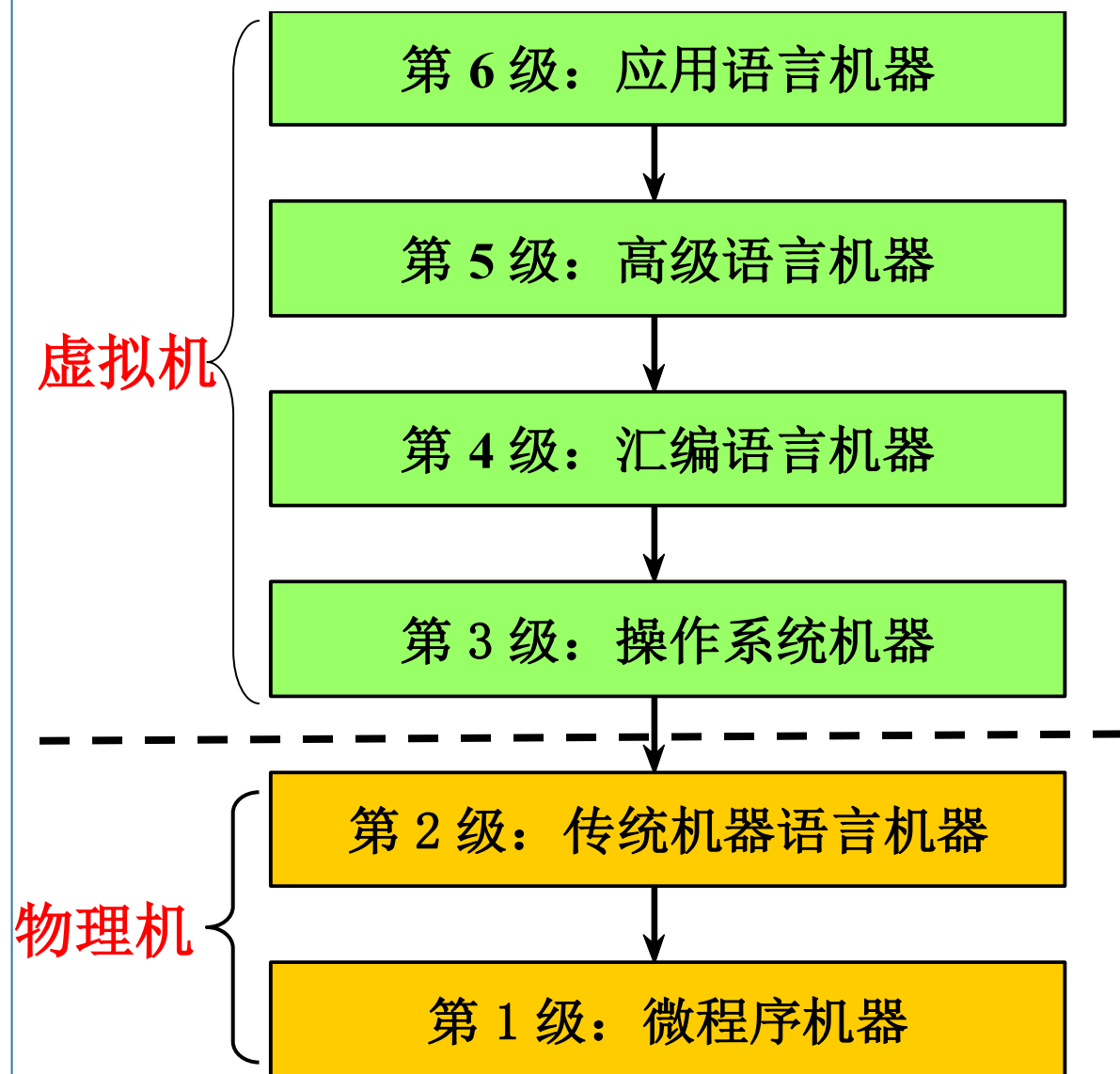
- 计算机系统 = 硬件 + 软件
- 计算机语言从低级向高级发展
 - ✓ 高一级语言的语句相对于低一级语言来说功能更强。
 - ✓ 更便于应用，但又都以低级语言为基础。
- 从计算机语言的角度，把计算机系统按功能划分成多级层次结构。
 - ✓ 每一层以一种语言为特征。

层次	层级名称	特征语言
第 6 层	高级语言层	C/C++、 Python、Java
第 5 层	汇编语言层	汇编语言
第 4 层	操作系统层	操作系统语言 / 系统调用
第 3 层	机器语言层 (指令集架构 层 ISA)	CPU 机器指令 (ARM、 LoongArch、 x86、RISC-V)
第 2 层	微程序层 (控 制层)	微指令
第 1 层	硬件逻辑层	

>>> 1.1 计算机系统的结构的基本概念

1.1.1 计算机系统的层次结构

- 物理机：用硬件实现的机器
- 虚拟机：由软件实现的机器（实际就是软件）
- 透明性：在计算机技术中，把这种本来存在的事物或属性，但从某种角度看好像又不存在的概念
- 语言实现的两种技术：各机器级的实现主要靠翻译或解释，或两者的结合
 - ✓ 翻译：先全部翻译转换（例如词法分析—语法分析—代码优化-存储分配—代码生成-目标程序）再执行。执行时间快（例如可重复）
 - ✓ 解释程序：源程序输入，执行时逐句翻译并执行（无目标程序）。执行时间慢，占用内存少。



➤ 计算机 system 结构 **经典定义**

- ✓ 传统机器程序员所看到的计算机属性，即概念性结构与功能特性（CPU 有哪些指令、寄存器数量（名字）、内存寻址方式等）。
- ✓ 所谓概念性结构与功能特性，实际上就是计算机 system 的外特性（简单说：ISA 指令集架构）。
- ✓ **这里的程序员**：机器语言、操作系统、汇编语言、编译程序的程序员。
- ✓ **看到的**：编写出能够在机器上正确运行的程序所必须了解到的东西。

➤ 功能特性，例如：

- ✓ (1) **数据表示**：硬件能直接识别和处理的数据类型和格式等的数据表示；
- ✓ (2) **寻址规则**：最小可寻址单位、寻址种类、地址计算等的寻址方式；
- ✓ (3) **寄存器定义**：通用/专用寄存器的设置、数量、字长、使用约定等的寄存器组织；
- ✓ (4) **指令集等**：二进制或汇编级指令的操作类型、格式、排序方式、控制机构等的指令系统；
- ✓ (5) **存储系统**：内存的最小编址单位、编址方式、容量、最大可编址空间等的存储系统组织；
- ✓ (6) **中断系统**：中断的分类与分级、中断处理程序功能及入口地址等的中断机构；
- ✓ (7) **I/O 结构**：输入输出设备的连接、使用方式、流量、操作结束、出错指示等的机器级 I/O 结构；

>>> 1.1 计算机系統結構的基本概念

1.1.2 计算机系統結構的定义

➤ 计算机系統結構的实质（机器能做什么，取决于指令集）

✓ 确定计算机系统中软硬件的界面，界面之上是软件实现的功能，界面之下是硬件和固件实现的功能。

✓ **指令集结构：用硬件实现的指令集**，软硬件之间的主要界面

以 ISA 为软硬件接口，在功耗、面积、成本约束下，通过硬件组织设计，挖掘各类并行性，利用存储层次，定量地提升程序执行性能；全程不断做各种权衡。

上下贯通
上知天文
下知地理

软件

硬件

指令集 (instruction set)



➤ 广义的系统结构定义：指令系统结构、组成、硬件（微体系结构）

✓ 计算机设计的三个方面，提出了对计算机系統結構的量化方法

>>> 1.1.3 计算机组成和计算机实现

➤ 计算机组成：计算机系统结构的逻辑实现

- ✓ 包含物理机器级中的数据流和控制流的组成以及逻辑设计等。
- ✓ 着眼于：物理机器级内各事件的排序方式与控制方式、各部件的功能以及各部件之间的联系。
- ✓ 计算机组成设计要解决的问题是在所希望达到的性能和价格，怎样最佳、最合理地把各种设备和部件组织成计算机，以实现所确定的系统结构。

➤ 计算实现：计算机组成的物理实现（如何制造）

- ✓ 包括处理机、主存等部件的物理结构，器件的集成度和速度，模块、插件、底板的划分与连接，信号传输，电源、冷却及整机装配技术等。
- ✓ 着眼于：器件技术（起主导作用）、微组装技术。

>>> 计算机系统结构、计算机组成和计算机实现

➤ 三者关系

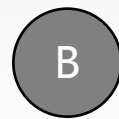
- ✓ 相同的系统结构（ISA），计算机组成可以不同
- ✓ 一种组成可以有多种不同的实现方法
- ✓ 采用不同的系统结构会使采用不同组成技术有所差异
- ✓ 组成也会影响结构

名称	别称	关注点
计算机系统结构	外特性、ISA 指令集	机器能做什么（功能）
计算机组成	内特性	如何实现这些功能
计算机实现	硬件物理实现	怎么造出硬件

1、指令系统的**确定**属于（ ）



计算机系统结构



计算机组成



计算机实现



其他

提交

2、指令的实现（取指令、分析指令、取操作数等）属于（ ）

A

计算机系统结构

B

计算机组成

C

计算机实现

D

其他

提交

>>> 1.1.4 计算机系统结构的分类

➤ 常见的**计算机系统结构分类法**有3种（根据一定的评价标准，将计算机分类）：

✓ Flynn分类法、冯氏分类法和Handler分类法。

➤ Flynn分类法（角度出发：同一时间，有几条指令在跑？同时处理几份数据？（根据指令流、数据流的数量去分类））

✓ 按照指令流和数据流的多倍性进行分类

- 指令流：计算机执行的指令序列
- 数据流：由指令流调用的数据序列
- 多倍性：在系统最受限的部件上，同时处于同一执行阶段的指令或数据的最大数目。

>>> 1.1.4 计算机系统的分类

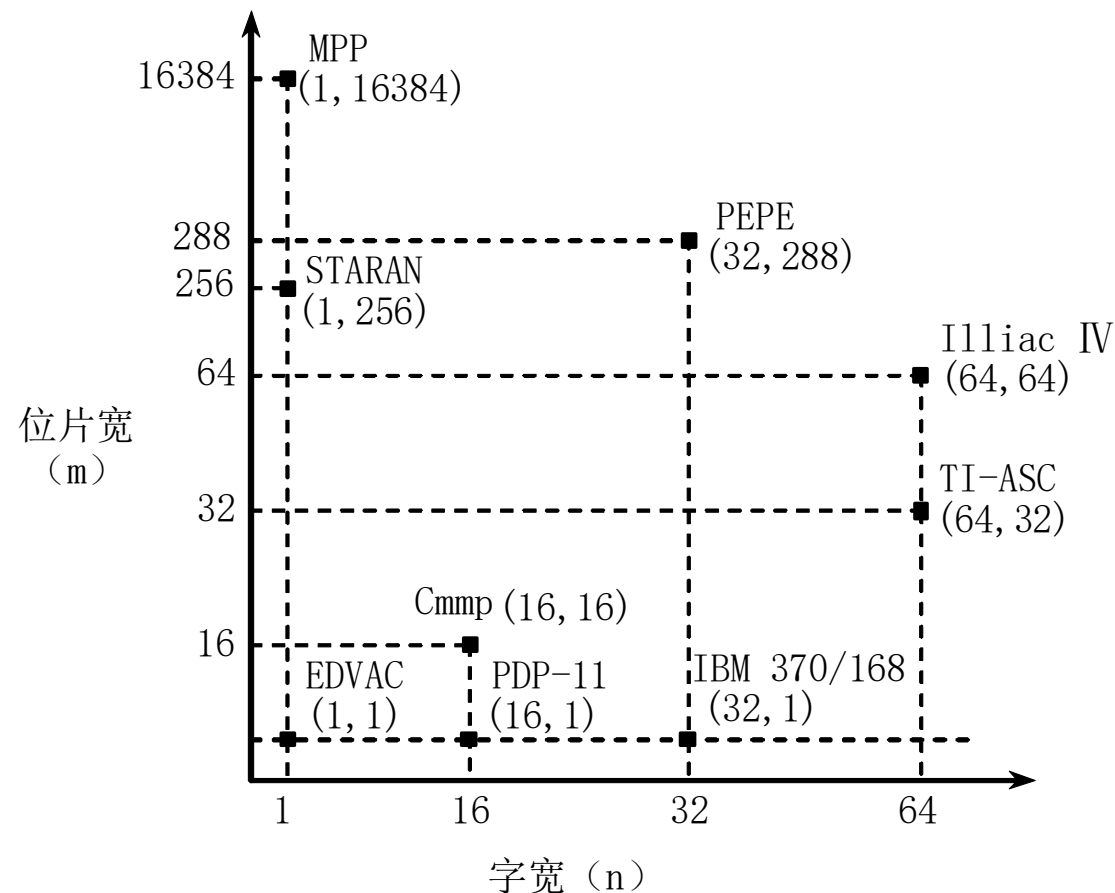
➤ Flynn分类法：把计算机系统的结构分为4类

单指令流单数据流SISD (Single Instruction stream Single Data stream) --传统串行单处理器计算机， --主要应用于流水技术	多指令流单数据流MISD (Multiple Instruction stream Single Data stream) --多条不同指令处理一组数据， 数据流计算机为代表，数据就绪才可使用，以数据为驱动
单指令流多数据流SIMD (Single Instruction stream Multiple Data stream) ---以向量处理机为代表，向量中的元素以空间并行	多指令流多数据流MIMD (Multiple Instruction stream Multiple Data stream) ---以多核处理器、多处理器、多计算机为代表
最终，都聚焦于并行	

>>> 1.1.4 计算机系统的分类

➤ **冯氏分类法**（评价角度：一次最多可以并行处理多少位**数据**（最大并行度））

- ✓ 用系统的最大并行度对计算机进行分类。
- ✓ 最大并行度：计算机系统在单位时间内能够处理的最大的二进制位数。
 - 用平面直角坐标系中的一个点代表一个计算机系统，其横坐标表示字宽（n位），纵坐标表示一次能同时处理的字数（m字）。 $m \times n$ 就表示了其最大并行度。



>>> 1.1.4 计算机系统的分类

➤ 冯氏分类法（从数据角度出发评价）

✓ 4类不同最大并行度的计算机系统结构

- 字串位串： $n = 1, m = 1$ 。（第一代计算机发展初期的纯串行计算机）
- 字串位并： $n > 1, m = 1$ 。这是传统的单处理机，同时处理单个字的多个位，如16位、32位等。
- 字并位串： $n = 1, m > 1$ 。同时处理多个字的同一位（位片）。
- 字并位并： $n > 1, m > 1$ 。同时处理多个字的多个位

✓ 平均并行度

- 与最大并行度密切相关的一个指标。
- 取决于系统的运用程度，与应用程序有关。

>>> 1.1.4 计算机系统结构的分类

➤ 冯氏分类法

假设每个时钟周期内能同时处理的二进制位数为 P_i ，则 T 个时钟周期内的平均并行度为：

$$P_a = \frac{\sum_{i=1}^T P_i}{T}$$

系统在 T 个时钟周期内的平均利用率定义为：

$$\mu = \frac{P_a}{P_m} = \frac{\sum_{i=1}^T P_i}{TP_m}$$

>>> 1.1.4 计算机系统结构的分类

➤ Handler分类法（评价角度：从硬件角度看并行能力）

✓ 根据并行度和流水线对计算机进行分类。

✓ 把计算机的硬件结构分成3个层次（统计每层最多多少部件并行工作）

- 程序控制部件（PCU）的个数 k
- 算术逻辑部件（ALU）或处理部件（PE）的个数 d 。
- 每个算术逻辑部件包含基本逻辑线路(ELC)的套数 w 。

✓ 用公式表示

- t （系统型号） = (k, d, w)

✓ 进一步改进

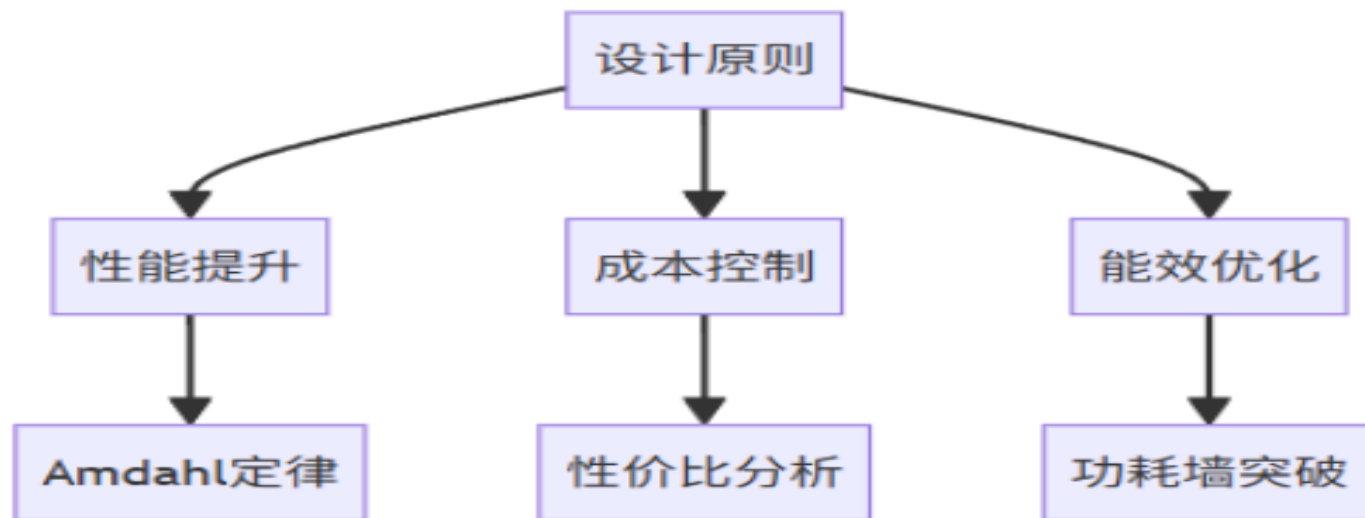
- t （系统型号） = $(k \times k', d \times d', w \times w')$

- k' ：宏流水线中程序控制部件的个数
- d' ：指令流水线中算术逻辑部件的个数
- w' ：操作流水线中基本逻辑线路的套数

>>> 1.2 计算机系统的设计

结合系统结构-组成-实现

- 计算机系统结构设计（顶层，ISA 指令集）：先定**外特性**：程序员能看见什么
- 计算机组成设计：内部实现方案（流水线几级、Cache多大、映射方式等）
- 计算机实现（硬件物理设计）



➤ 4个定量原理：

- ✓ 以经常性事件为重点
- ✓ Amdahl定律
- ✓ CPU性能公式
- ✓ 程序的局部性原理

➤ 目标：如何优化上述4个定量操作提高计算机性能

➤ 以经常性事件为重点

- ✓ 定性思路：经常发生的事件，优化它收益更大。
- ✓ 定量的体现：配合定量计算才能权衡，
- ✓ 例如统计定量统计：事件出现的占比、耗时占比；然后用 Amdahl 定律、CPU 性能公式去量化评估优化带来的性能提升；

➤ 程序的局部性原理

- ✓ 定性思路：程序访问内存不是均匀随机的，存在时间局部性、空间局部性；
- ✓ 定量的体现：用数值指标量化局部性强弱；
- ✓ 例如：cache命中率、访问地址分布统计、重用距离（两次访问同一个地址相隔多少指令）

➤ 4个定量原理:

✓ 以经常性事件为重点

✓ Amdahl定律

✓ CPU性能公式

✓ 程序的局部性原理

✓ 以经常性事件为重点:对经常发生的情况采用优化方法的原则进行选择, 以得到更多的总体上的改进。

优化是指分配更多的资源、达到更高的性能或者分配更多的电能等。

✓ 程序的局部性原理:程序执行时所访问的存储器地址分布不是随机的, 而是相对地簇聚 (程序执行时间的90%都是在执行程序中10%的代码)。

程序的时间局部性: 程序即将用到的信息很可能就是目前正在使用的信息。

程序的空间局部性: 程序即将用到的信息很可能与目前正在使用的信息, 在空间上相邻或者临近。

>>> 1.2 计算机系统的设计

1.2.1 计算机系统设计的定量原理

➤ 4个定量原理:

- ✓ 以经常性事件为重点
- ✓ Amdahl定律
- ✓ CPU性能公式
- ✓ 程序的局部性原理

✓ 以经常性事件为重点: 优化时允许让不常发生的事件变慢, 只要常见事件大幅提速, 整体性能提升;

✓ 本质: 权衡, 体系结构最基础设计思想; 筛选候选优化对象, 为后面阿姆达尔 / CPU 性能公式的计算定方向。

✓ 例如: Cache 优化访存 (大多数访问命中 Cache, 少数访问内存)、分支预测 (大多数分支可以预测正确)

加快经常事件, 整体事件
时间减少很多

例如: 浮点运算占程序总指令 25%, 访存指令占 50%。访存是经常性事件 → 优先优化存储器、增加 Cache, 而不是花大力气优化很少用到的除法指令。

✓ 蓝色代表经常事件, 红色代表非经常事件



✓ 加快非经常事件时间10倍



✓ 加快经常事件时间5倍

>>> 1.2 计算机系统的设计

1.2.1 计算机系统设计的定量原理

➤ 4个定量原理:

- ✓ 以经常性事件为重点
- ✓ Amdahl定律
- ✓ CPU性能公式
- ✓ 程序的局部性原理

加快经常事件，整体事件时间减少很多

✓ 以经常性事件为重点: 优化时允许让不常发生的事件变慢，只要常见事件大幅提速，整体性能提升;

✓ 本质: 权衡，体系结构最基础设计思想; 筛选候选优化对象，为后面阿姆达尔 / CPU 性能公式的计算定方向。

✓ 例如: Cache 优化访存 (大多数访问命中 Cache, 少数访问内存)、

✓ 分支预测 (大多数分支可以预测正确)
✓ : 浮点运算占程序总指令 25%, 访存指令占 50%。

访存是经常性事件 → 优先优化存储器、增加 Cache,

✓ 蓝色代表经常事件，红色代表非经常事件
而不是花大力气优化很少用到的除法指令。



✓ 加快非经常事件时间10倍

✓ 加快经常事件时间5倍

- 4个定量原理：
- ✓以经常性事件为重点

✓Amdahl定律

✓CPU性能公式

✓程序的局部性原理
- ✓程序的局部性原理:程序执行时所访问的存储器地址分布不是随机的，而是相对地簇聚（程序执行时间的90%都是在执行程序中10%的代码）。

✓ 优化策略：cache引入，提高内存和CPU之间的访问效率

✓ 局部性实例：例如for循环，

✓ 局部性事件：指令局部性、数据局部性、转移局部性（同一条指令常往同一个方向）、cache、预期、转移预测等

	时间局部性	空间局部性
指令	循环语句指令	顺序语句指令
数据	循环变量i	字符串、数组

>>> 1.2.2 计算机系统的设计

➤ Amdahl定律（阿姆达尔定律）

- ✓ 加快某部件执行速度所能获得的系统性能加速比，受限于该部件的执行时间占系统中总执行时间的百分比。（定量指标衡量优化前后的性能变化）

➤ 对改进部分的性能进行量化

➤ 系统性能加速比（定量指标）：

能否一次
优化全部
部件？

$$\text{加速比} = \frac{\text{系统性能}_{\text{改进后}}}{\text{系统性能}_{\text{改进前}}} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$

>>> 1.2.2 计算机系统的设计

- Amdahl定律（阿姆达尔定律）：系统性能加速比（定量指标）

$$\text{加速比} = \frac{\text{系统性能}_{\text{改进后}}}{\text{系统性能}_{\text{改进前}}} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$

- 加速比依赖于两个因素

- ✓ 可改进比例（ F_e ）：在改进前的系统中，可改进部分的执行时间在总的执行时间中所占的比例。
它总是小于等于1。

例如：一个需运行60秒的程序中有20秒的运算可以加速，那么这个比例就是20/60。

- ✓ 部件加速比（ S_e ）（引入该参数）：可改进部分改进以后性能提高的倍数。它是改进前所需的执行时间与改进后执行时间的比，即改进后自身可以加快的倍数。

一般情况下部件加速比是大于1的。

例如：若系统改进后，可改进部分的执行后时间是2秒，而改进前其执行时间为5秒，则部件加速比为5/2。

>>> 1.2.2 计算机系统的设计

➤ Amdahl定律 (阿姆达尔定律)

✓ 改进前程序的总执行时间 T_0



✓ 改进后程序的总执行时间 T_n

✓ 改进后程序的总执行时间 T_n

$$T_n = T_0 \left(1 - F_e + \frac{F_e}{S_e} \right)$$

□ T_0 : 改进前整个程序的执行时间

□ $1 - F_e$: 不可改进比例, F_e 改进比例, $\frac{F_e}{S_e}$ 缩短的倍数

➤ 系统加速比, S_n 为改进前与改进后总执行时间之比

$$S_n = \frac{T_0}{T_n} = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}}$$

>>> 1.2.2 计算机系统的设计者

✓ Amdahl定律:

✓ Amdahl定律: 一种性能改进的递减规则

- 如果仅仅对计算任务中的一部分做性能改进, 则改进得越多, 所得到的总体性能的提升就越有限。

✓ 重要推论: 如果只针对整个任务的一部分进行改进和优化, 那么所获得的加速比不超过:

$$1 / (1 - \text{可改进比例})$$

例1.1 将计算机系统中某一功能的处理速度加快15倍, 但该功能的处理时间仅占整个系统运行时间的40%, 则采用此增强功能方法后, 能使整个系统的性能提高多少?

解 由题可知: $F_e = 40\% = 0.4$

$$S_e = 15$$

根据Amdahl定律可知:

$$S_n = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}} = \frac{1}{(1 - 0.4) + \frac{0.4}{15}} \approx 1.6$$

采用此增强功能方法后, 能使整个系统的性能提高到原来的1.6倍。

>>> 1.2.2 计算机系统的设计

✓ Amdahl定律:

例1.2 某计算机系统采用浮点运算部件后，使浮点运算速度提高到原来的25倍，而系统运行某一程序的整体性能提高到原来的4倍，试计算该程序中浮点操作所占的比例。

解 由题可知： $S_e = 25$ $S_n = 4$

根据Amdahl定律可知：

$$4 = \frac{1}{(1 - Fe) + \frac{Fe}{25}}$$

由此可得： $Fe = 78.1\%$

即程序中浮点操作所占的比例为78.1%。

>>> 1.2.2 计算机系统的设计

➤ CPU性能公式

- ✓ 执行一个程序所需的CPU时间

CPU时间 = 执行程序所需的时钟周期数 × 时钟周期时间

其中：时钟周期时间是系统时钟频率的倒数。

- ✓ 每条指令执行的平均时钟周期数CPI (Cycles Per Instruction)

$$CPI = \text{执行程序所需的时钟周期数} / IC$$

IC: 所执行的指令条数

- ✓ 程序执行的CPU时间可以写成

$$\text{CPU时间} = IC \times CPI \times \text{时钟周期时间}$$

>>> 1.2.2 计算机系统的设计

➤ CPU性能公式

✓ CPU的性能取决于三个参数（拆解 CPU 时间由哪几个因素决定；用来分析性能瓶颈在哪）

- 时钟周期时间：取决于硬件实现技术和计算机组成；
- CPI：取决于计算机组成和指令系统的结构（例如要不要流水线技术、要不要cache技术？要与不要对CPI的影响）；
- IC：取决于指令系统的结构和编译技术。

✓ 对CPU性能公式进行进一步细化

假设：计算机系统有n种指令；

CPI_i ：第i种指令的处理时间；

IC_i ：在程序中第i种指令出现的次数；

则：

$$\text{CPU时钟周期数} = \sum_{i=1}^n (CPI_i \times IC_i)$$

>>> 1.2.2 计算机系统的设计

➤ CPU性能公式

CPU时间 = 执行程序所需的时钟周期数 × 时钟周期时间

$$= \sum_{i=1}^n (CPI_i \times IC_i) \times \text{时钟周期时间}$$

CPI可以表示为：

$$CPI = \frac{\text{时钟周期数}}{IC} = \frac{\sum_{i=1}^n (CPI_i \times IC_i)}{IC} = \sum_{i=1}^n (CPI_i \times \frac{IC_i}{IC})$$

其中：(IC_i/IC)反映了第*i*种指令在程序中所占的比例。

IC_i 为程序执行过程中第*i*种指令出现的次数

CPI_i 为程序执行第*i*种指令所需的平均时钟周期数，*n*为指令种数

>>> 1.2.2 计算机系统设的计

➤ CPU性能公式

例1.3 假设FP（浮点数操作）指令的比例为25%，其中，FPSQR（浮点数平方根操作）占全部指令的比例为2%，FP操作的CPI为4，FPSQR操作的CPI为20，其他指令的平均CPI为1.33。现有两种改进方案，第一种是把FPSQR操作的CPI减至2，第二种是把所有的FP操作的CPI减至2，试比较两种方案对系统性能的提高程度。

解 没有改进之前，每条指令的平均时钟周期CPI为：

$$CPI = \sum_{i=1}^n \left(CPI_i \times \frac{IC_i}{IC} \right) = (4 \times 25\%) + (1.33 \times 75\%) = 2$$

指令类型	FP		其他
	FPSQR	其它FP	
CPI	20	未知	1.33
	所有FP的平均CPI为4		
百分比	2%	25%- 2%=23%	75%
	共25%		

>>> 1.2.2 计算机系统的设计

➤ CPU性能公式

(1) 采用第一种方案

节省的CPI

FPSQR操作的CPI由 $CPI_{FPSQR}=20$ 减至 $CPI'_{FPSQR}=2$ ，
则整个系统的指令平均时钟周期数为 CPI_1 ：

$$CPI_1 = CPI_{原} - (CPI_{FPSQR} - CPI'_{FPSQR}) \times 2\% \\ = 2 - (20 - 2) \times 2\% \approx 1.64$$

或其他计算方法(将FPSQR和其他FP看成一个整体)

除了FPSQR指令其他FP指令的平均CPI为M，

则 $M \times 0.98 + 20 \times 0.02 = 2$ $M \approx 1.63$ $CPI_1 = 1.33 \times 0.75 + 1.63 \times 0.23 + 2 \times 0.02 \approx 1.64$

FPSQR操作的CPI为20

指令类型	FP		其他
	FPSQR	其它FP	
CPI	20	未知	1.33
百分比	2%	25%-2%=23%	75%
	共25%		

改进FPSQR

已知FPSQR指令和除了FPSQR之外的FP指令和其他指令的比例，还有FPSQR指令和其他指令的CPI，缺少的是除了FPSQR之外的其他FP指令的平均CPI，因此要先求出除了FPSQR指令外其他FP指令的平均CPI，除了FPSQR指令外其他FP指令的平均CPI可以通过比例算出

>>> 1.2.2 计算机系统的设计

➤ CPU性能公式

改进FP 操作，
已知其他指令的平均CPI和改进后FP
操作的平均CPI，两者各自的比例

指令类型	FP		其他
	FPSQR	其它FP	
CPI	20	未知	1.33
	所有FP的平均CPI为4		
百分比	2%	25%- 2%=23%	75%
	共25%		

(2) 采用第二种方案

所有FP操作的CPI由CPI_{FP}=4减至CPI'_{FP}=2，则整个系统的指令平均时钟周期数为(FPSQR操作不改变):

$$\begin{aligned} \text{CPI}_2 &= \text{CPI} - (\text{CPI}_{\text{FP}} - \text{CPI}'_{\text{FP}}) \times 25\% \\ &= 2 - (4 - 2) \times 25\% = 1.5 \end{aligned}$$

或 其他方法: $\text{CPI}_2 = 0.75 \times 1.33 + 0.25 \times 2 = 1.5$

从降低整个系统的指令平均时钟周期数的程度来看，第二种方案优于第一种方案。

>>> 1.2.2 计算机系统设计者的主要任务

- 计算机系统设计者的任务包括：指令系统的设计、数据表示的设计、功能的组织、逻辑设计以及其物理实现等。
- 设计一个计算机系统大致要完成3个方面的工作。
 - ✓ 1、确定用户对计算机系统的功能、价格和性能的要求
 - ✓ 2、软硬件功能分配
 - ✓ 3、设计出生命周期长的系统结构

>>> 1.2.2 计算机系统设计者的主要任务

➤ 设计一个计算机系统大致要完成3个方面的工作

✓ 1、确定用户对计算机系统的功能、价格和性能的要求

- 计算机系统设计者的目标：设计出能满足用户的功能需求、有较长的生命周期、且又具有很高的性能价格比的系统；
- 功能需求：根据市场的需要以及所设计系统的应用领域来确定（应用领域、软件兼容、操作系统需求、标准等）；
- 三大指标：功能（例如需说明的op、ISA等）、价格、性能（用什么评估）。

>>> 1.2.2 计算机系统设计者的主要任务

➤ 设计一个计算机系统大致要完成3个方面的工作。

✓ 2、软硬件功能分配

- 考虑如何优化设计？ 必须考虑软硬件功能的合理分配。
- 软件和硬件在实现功能上是等价的 用软件实现的优点：设计容易、修改简单，而且可以减少硬件成本。但是所实现的功能的速度较慢。用硬件实现的优点：速度快、性能高，但它修改困难，灵活性差。
- 在软硬件之间进行折中和取舍（例如加快经常性事件）。

>>> 1.2.2 计算机系统设计者的主要任务

➤ 设计一个计算机系统大致要完成3个方面的工作。

✓ 3、设计出生命周期长的系统结构

- 特别注意计算机应用和计算机技术的发展趋势
- 设计出具有一定前瞻性的系统结构，以使得它具有较长的生命周期。

三大任务

先确定用户需要什么 → 然后决定哪些硬件做、哪些软件做 → 最后设计一套稳定、可扩展、生命周期长的架构，方便后续迭代升级

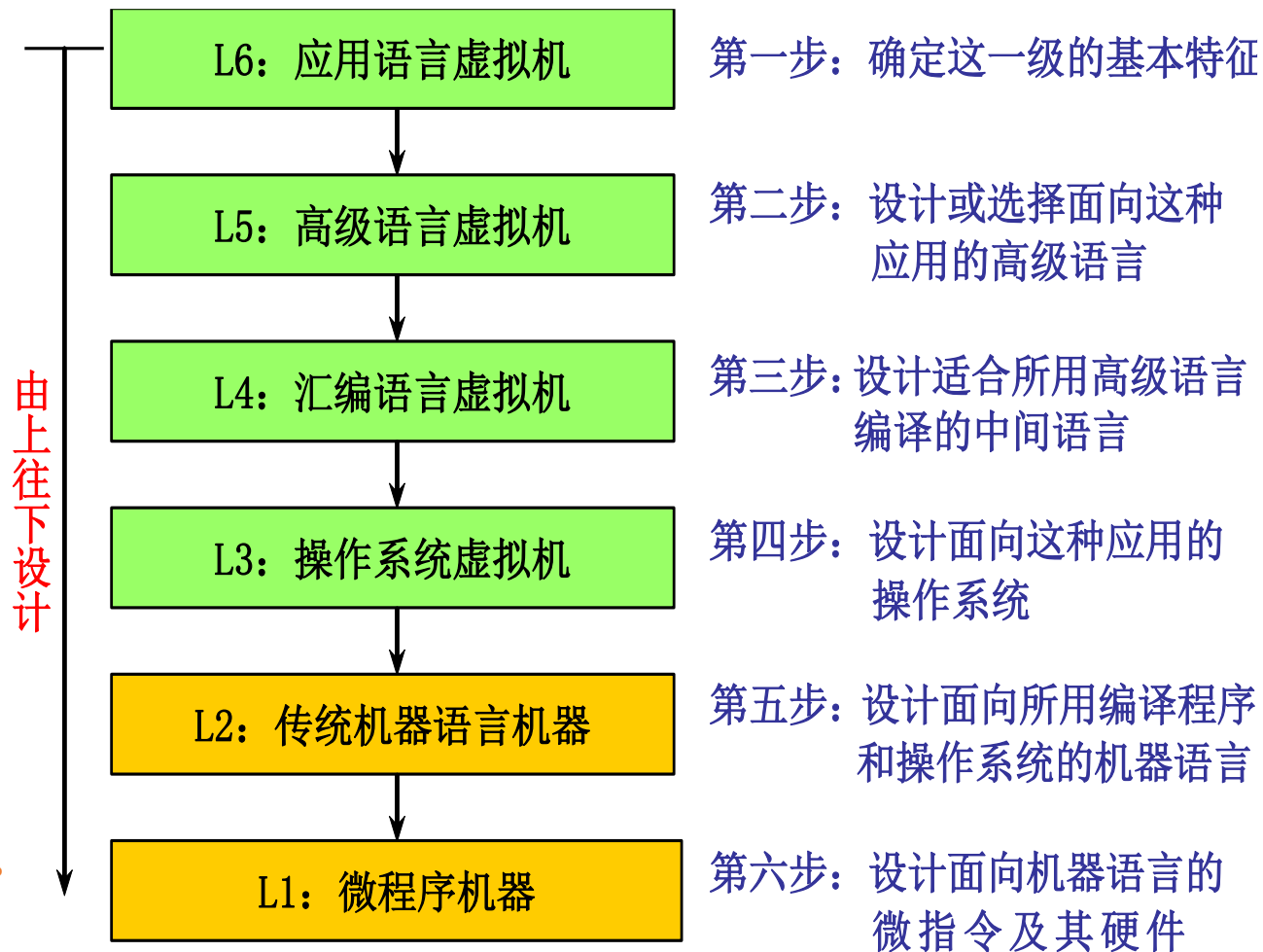
>>> 1.2.2 计算机系统设计者的主要方法

****用什么方法去完成上述三大任务**。**

➤ “由上往下” (top-down) 设计

✓ 从层次结构中的最上面一级开始，逐层往下设计各层的机器

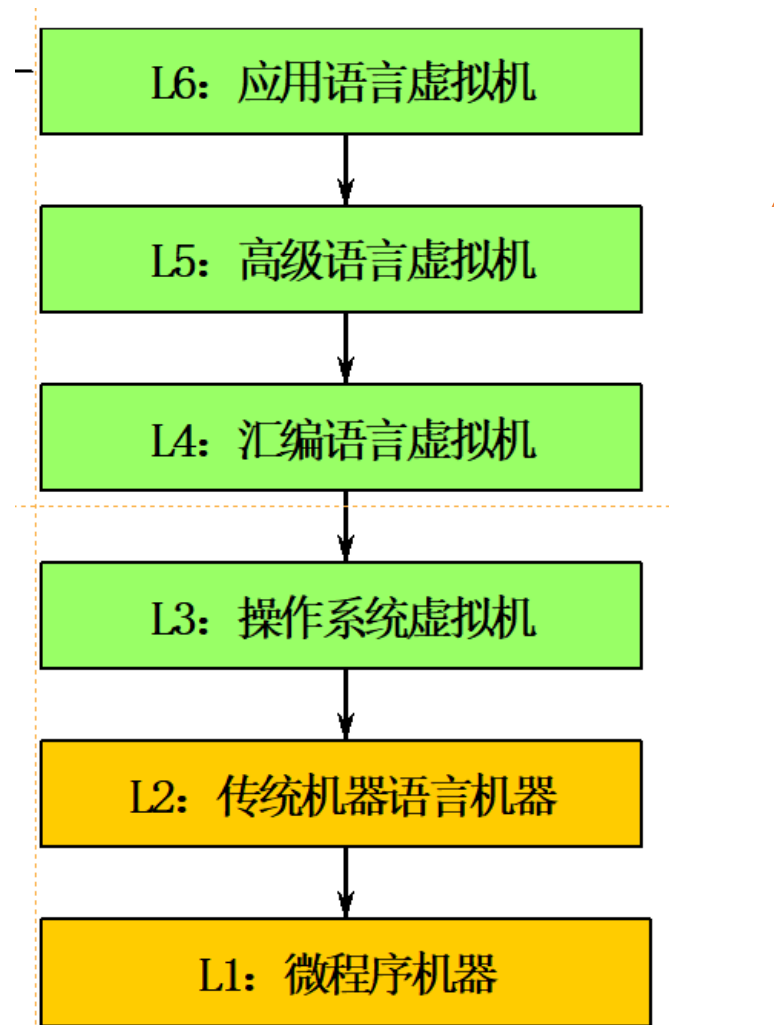
- 首先确定面对使用者的那级机器的基本特征、数据类型和格式、基本命令等。
- 然后再逐级往下设计，每级都考虑如何优化上一级的实现。（一旦上层应用需求改变，下面整套软硬件都可能要重新设计）
- 适合于专用机的设计，而不适合通用机的设计。



>>> 1.2.2 计算机系统设计者的主要方法

➤ “由下往上” (bottom-up) 设计

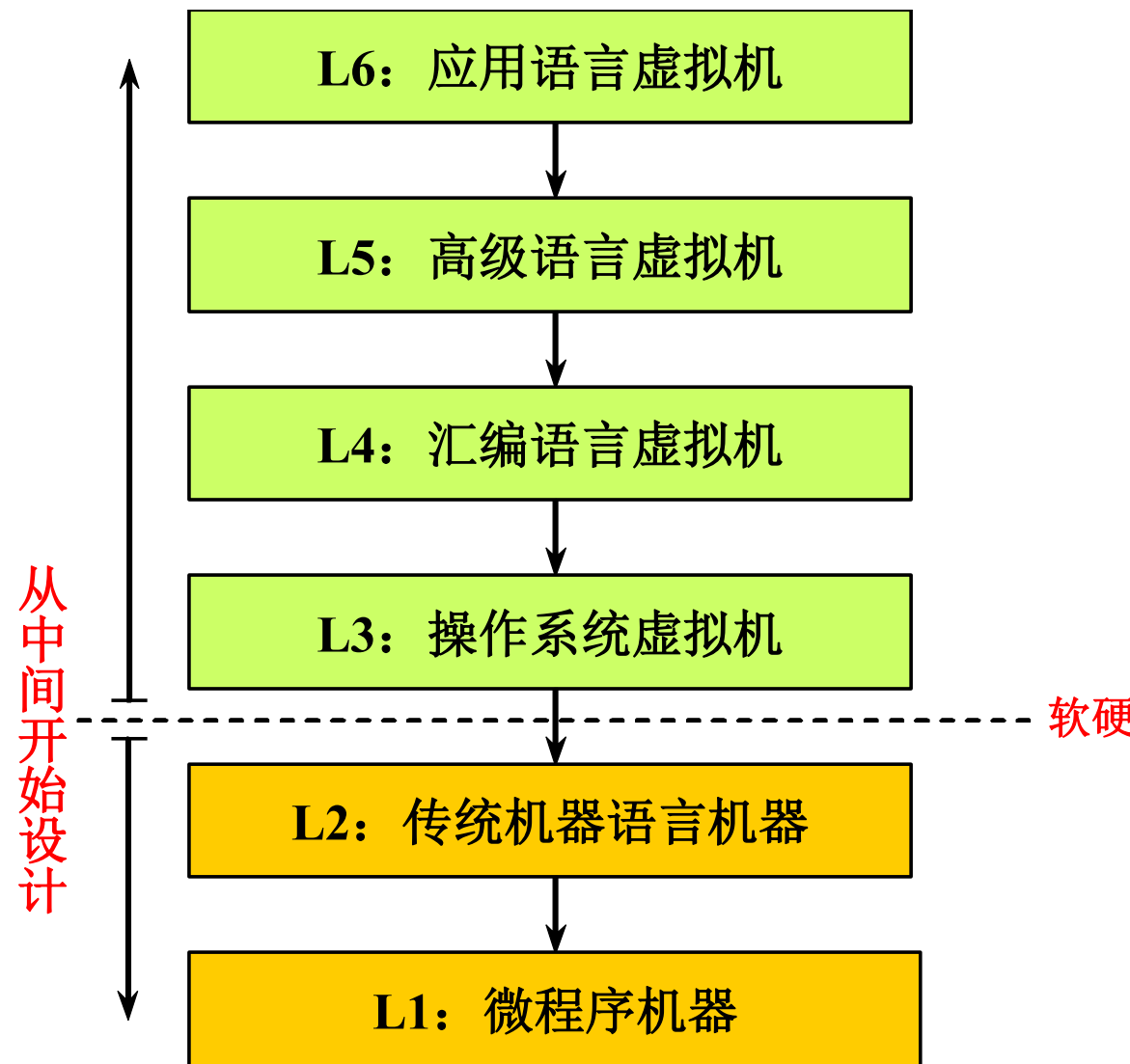
- ✓ 从层次结构的最下面一级开始，逐层往上设计各层的机器。（**底层**：晶体管、逻辑门、ALU、寄存器、功能部件；**往上**：数据通路、控制器 → CPU → 整机系统。）
- ✓ 每一层的任务就是：如何实现、优化上一层的功能。
- ✓ 采用这种方法时，**硬件先行，再说软件是否适应软件**；**软件技术完全处于被动状态，这会造成软件和硬件的脱节**，使整个系统的效率降低。
(在早期被采用得比较多，现在已经很少被采用了)



>>> 1.2.2 计算机系统设计者的主要方法

➤ "从中间开始" (middle-out) 设计

- ✓ “由上往下”和“由下往上”设计方法的主要缺点
软、硬件设计分离和脱节
- ✓ 解决方法：综合考虑软、硬件的分工，从中间开始设计。
 - “中间”：层次结构中的软硬件的交界面，目前一般是在传统机器语言机器级与操作系统机器级之间。
- ✓ 从中间开始设计
 - 首先要进行软、硬件功能分配，确定好这个界面。
 - 然后从这个界面开始，软件设计者开始往上设计操作系统、汇编、编译系统等，硬件设计者开始往下设计传统机器级、微程序机器级等。



➤ 评测标准

- ✓ 不能只看单个指标，用程序说话-----真实的应用程序（序列基准程序）
- ✓ 计算机系列不同，侧重不同，测试程序也不同
- ✓ 对测序程序要求：典型，评价全面，但可能不会专注细节
- ✓ 不同测试程序测试，需要总体如何量化，例如平均？归一化？取值范围？等
- ✓ 完备的测序文档：详细，操作可复盘

>>> 1.3 计算机系统的性能评测（注重三个方面）

➤ 系统性能评测：执行时间和吞吐率、基准测试程序、性能测试

➤ 1、执行时间和吞吐率

如何评测一台计算机的性能，与测试者看问题的角度有关。

- ✓ 用户关心的是：单个程序的执行时间（执行单个程序所花的时间很少）
- ✓ 数据处理中心的管理人员关心的是：吞吐率（在单位时间里能够完成的任务很多）

>>> 1.3 计算机系统的性能评测

- 假设两台计算机为X和Y，X比Y快：

对于给定任务，X的执行时间比Y的执行时间少。

- X的性能是Y的n倍：

$$\frac{\text{执行时间Y}}{\text{执行时间X}} = n$$

- 执行时间与性能成反比：

$$n = \frac{\text{执行时间Y}}{\text{执行时间X}} = \frac{\frac{1}{\text{性能Y}}}{\frac{1}{\text{性能X}}} = \frac{\text{性能X}}{\text{性能Y}}$$

>>> 1.3 计算机系统的性能评测

1、执行时间和吞吐率

➤ 执行时间可以有多种定义：

- ✓ 计算机完成某一任务所花费的全部时间，包括磁盘访问、存储器访问、输入/输出、操作系统开销等。
- ✓ CPU时间：CPU执行所给定的程序所花费的时间，不包含I/O等待时间以及运行其它程序的时间。
 - 用户CPU时间：用户程序所耗费的CPU时间。
 - 系统CPU时间：用户程序运行期间操作系统耗费的 CPU时间。

>>> 1.3 计算机系统的性能评测

➤ 2、基准测试程序

- ✓ 用于测试和比较性能的基准测试程序的最佳选择是真实应用程序。（例如编译器）
- ✓ 以前常采用简化了的程序，例如：
 - 核心测试程序：从真实程序中选出的关键代码段构成的小程序。
 - 小测试程序：简单的只有几十行的小程序。
 - 合成的测试程序：人工合成出来的程序。

从测试性能的角度来看，上述测试程序不可信了。（程序小、片面性、人为性等）

>>> 1.3 计算机系统的性能评测

2、基准测试程序

除上述对测试程序的要求外，另外：

- ✓ 在什么条件下进行测试？如何测试-分类等？

基准测试程序设计者对制造商的要求

- ✓ 采用同一种语言编译器；
- ✓ 对同一种语言的程序都采用相同的一组编译标志。

使用经典的测试程序

- ✓ **基准测试程序套件：**由各种不同的真实应用程序构成。
(能比较全面地反映计算机在各个方面的处理性能)
- ✓ **SPEC系列。**

>>> 1.3 计算机系统的性能评测

3. 性能比较

两个程序在A、B、C三台机器上的执行时间

	机器A	机器B	机器C	W (1)	W (2)	W (3)
程序1	1.00	10.00	20.00	0.50	0.909	0.999
程序2	1000.00	10.00	20.00	0.50	0.091	0.001
加权算术 平均值A _m (1)	500.50	10.00	20.00			
加权算术 平均值A _m (2)	91.91	10.00	20.00			
加权算术 平均值A _m (3)	2.00	10.00	20.00			

三台机器的性能

从该表可以得出：

执行程序1：

- A机的速度是B机的10倍
- A机的速度是C机的20倍
- B机的速度是C机的2倍

执行程序2：

- B机的速度是A机的100倍
- C机的速度是A机的50倍
- B机的速度是C机的2倍

但不能综合去衡量三种机器的性能

>>> 1.3 计算机系统的性能评测

3. 性能比较

两个程序在A、B、C三台机器上的执行时间

	机器A	机器B	机器C	W (1)	W (2)	W (3)
程序1	1.00	10.00	20.00	0.50	0.909	0.999
程序2	1000.00	10.00	20.00	0.50	0.091	0.001
加权算术 平均值A _m (1)	500.50	10.00	20.00			
加权算术 平均值A _m (2)	91.91	10.00	20.00			
加权算术 平均值A _m (3)	2.00	10.00	20.00			

总执行时间：机器执行所有测试程序的总时间

B机执行程序1和程序2的速度是A机的50.05倍

C机执行程序1和程序2的速度是A机的24.02倍

B机执行程序1和程序2的速度是C机的2倍

平均执行时间：各测试程序执行时间的
算术平均值

$$S_m = \frac{1}{n} \sum_{i=1}^n T_i$$

其中：T_i：第i个测试程序的执行时间

n：测试程序组中程序的个数

>>> 1.3 计算机系统的性能评测

3. 性能比较

两个程序在A、B、C三台机器上的执行时间

	机器A	机器B	机器C	W (1)	W (2)	W (3)
程序1	1.00	10.00	20.00	0.50	0.909	0.999
程序2	1000.00	10.00	20.00	0.50	0.091	0.001
加权算术 平均值A _m (1)	500.50	10.00	20.00			
加权算术 平均值A _m (2)	91.91	10.00	20.00			
加权算术 平均值A _m (3)	2.00	10.00	20.00			

加权执行时间：各测试程序执行时间的加权
平均值

$$A_m = \sum_{i=1}^n W_i \cdot T_i$$

其中：T_i：第i个测试程序的执行时间
n：测试程序组中程序的个数

$$\sum_{i=1}^n W_i = 1$$

>>> 1.3 计算机系统的性能评测

3. 性能比较

两个程序在A、B、C三台机器上的执行时间

	机器A	机器B	机器C	W (1)	W (2)	W (3)
程序1	1.00	10.00	20.00	0.50	0.909	0.999
程序2	1000.00	10.00	20.00	0.50	0.091	0.001
加权算术 平均值A _m (1)	500.50	10.00	20.00			
加权算术 平均值A _m (2)	91.91	10.00	20.00			
加权算术 平均值A _m (3)	2.00	10.00	20.00			

加权几何平均值

$$G_m = \prod_{i=1}^n (R_i)^{W_i} = (R_1)^{W_1} \times (R_2)^{W_2} \times \cdots \times (R_n)^{W_n}$$

G_m表示法有一个很好的特性：几何平均值的比等于比的几何平均值

$$\frac{G_m(x_i)}{G_m(y_i)} = G_m\left(\frac{x_i}{y_i}\right)$$

>>> 1.3 计算机系统的性能评测

3. 性能比较

- ✓ **几何平均值法**：以某台计算机的性能作为参考标准，其他计算机性能则除以该参考标准而获得一个比值。

$$G_m = \sqrt[n]{\prod_{i=1}^n R_i} = \sqrt[n]{\prod_{i=1}^n \frac{1}{T_i}}$$

R_i ：由 n 个程序组成的工作负荷中执行第 i 个程序的速度

$$R_i = 1/T_i$$

$\prod$ ：连乘

➤➤➤ 1.2 计算机系统设计定量原理 VS 1.3 计算机系统的性能评测

➤ 4个定量原理:

- ✓ 以经常性事件为重点
- ✓ Amdahl定律
- ✓ CPU性能公式
- ✓ 程序的局部性原理

➤ **目标:** 预判如何优化上述4个定量操作提高计算机性能

➤ 系统设计阶段

➤ 3个方面展开:

- ✓ 执行时间和吞吐率
- ✓ 基准测试程序
- ✓ 性能测试

➤ **目标:** 检验系统真实性能, 横向对比不同成品 CPU

➤ 系统测试阶段

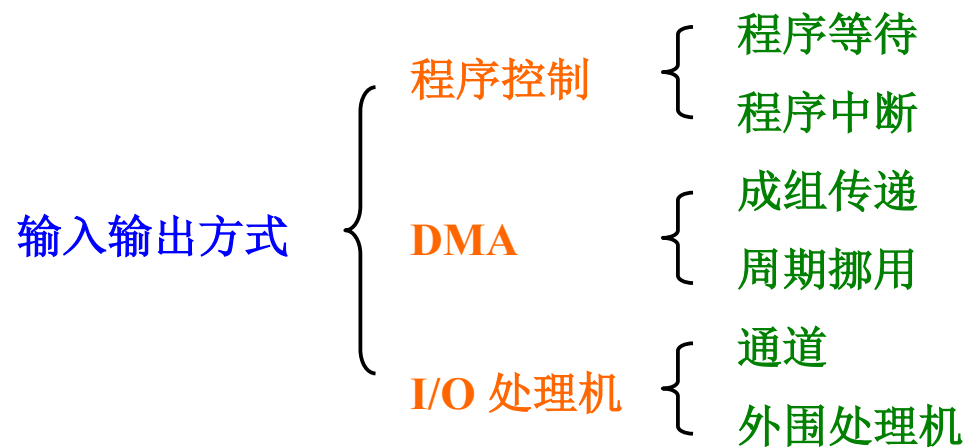
若测试阶段发现瓶颈问题, 反过来去优化下一阶段的设计



1.4.1 冯·诺依曼结构及其改进

➤ 3、对系统结构进行的改进（主要4个方面）

✓ 1、输入/输出方式的改进



✓ 2、采用并行处理技术

- 如何挖掘传统机器中的并行性？
- 在不同的级别采用并行技术。

例如：微操作级、指令级、线程级、进程级、任务级等。



1.4.1 冯·诺依曼结构及其改进

➤ 3、对系统结构进行的改进（主要4个方面）

✓ 3、存储器组织结构的发展

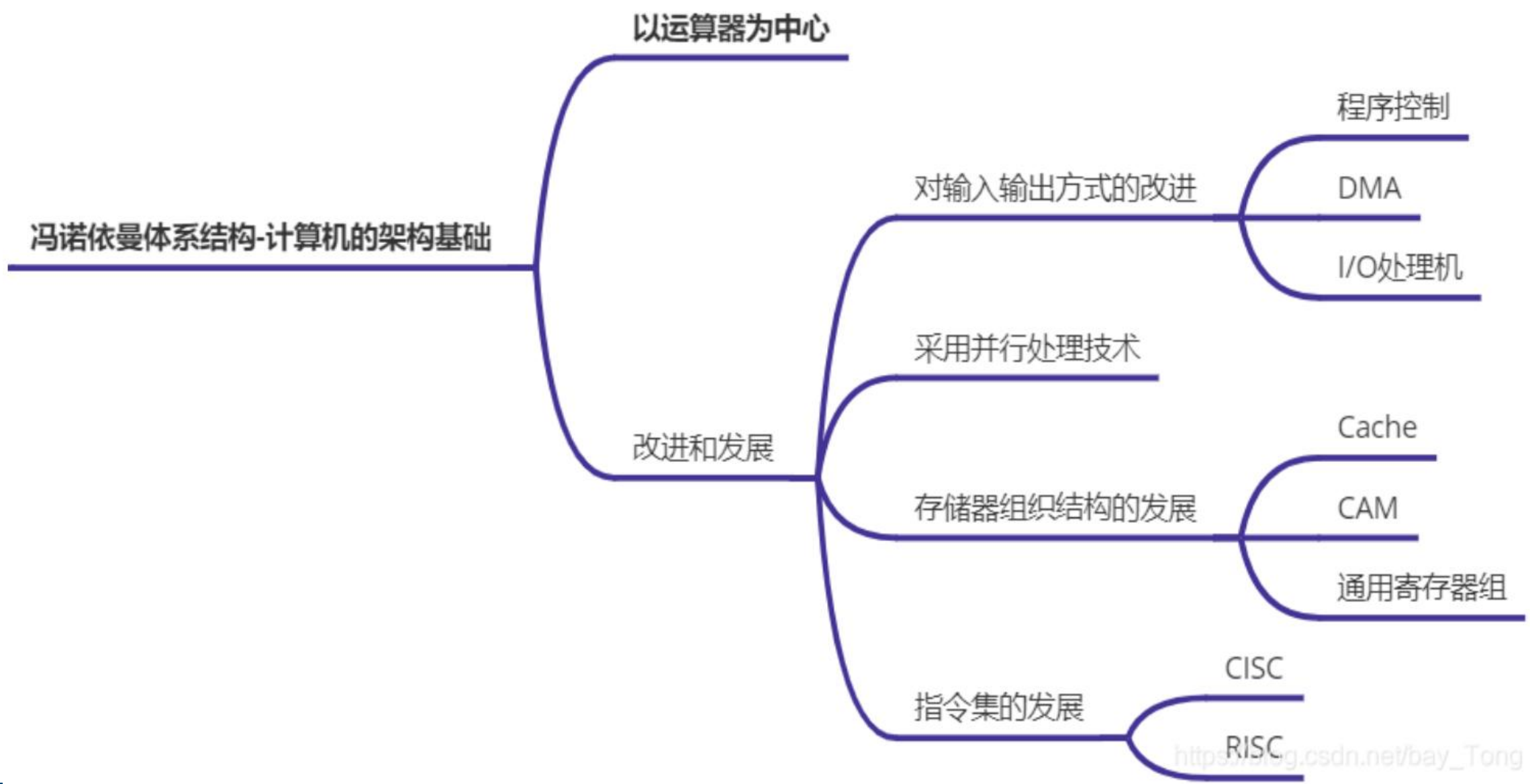
- 相联存储器与相联处理机 （传统存储查找：按照地址；相联存储器：按内容检索，速度快）
- 通用寄存器组 （不但可存储数据，还可参与算术逻辑计算，加速指令执行）
- 高速缓冲存储器Cache

✓ 4、指令系统的发展

两个发展方向：

- 复杂指令集计算机CISC
- 精简指令集计算机RISC

>>> 1.4.1 冯·诺依曼结构及其改进





1.4.2 软件对系统结构的影响

- **软件的可移植性：**一个软件可以不经修改或者只需少量修改就可以由一台机器移植到另一台机器上正确地运行。差别只是执行时间的不同。我们称这两台机器是**软件兼容**的。
-
- **实现可移植性的常用方法：**统一高级语言，采用系列机，模拟与仿真，

>>> 1.4.2 软件对系统结构的影响

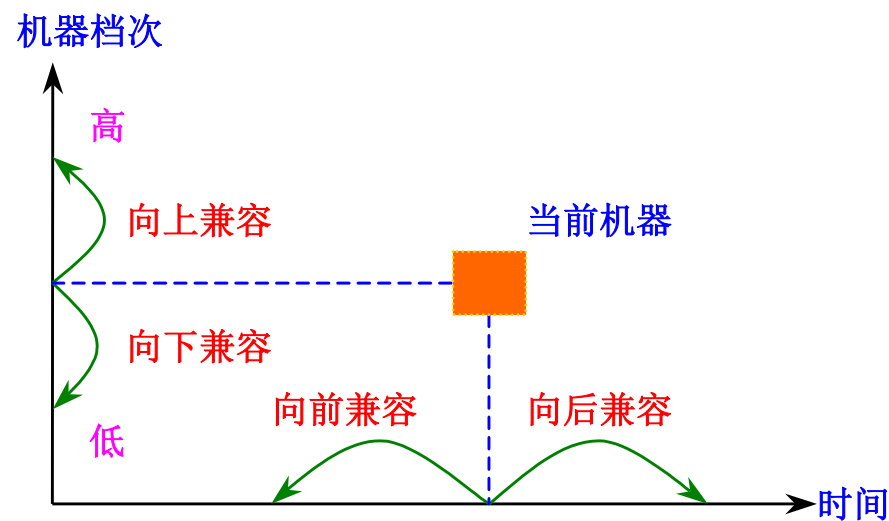
➤ 1、统一高级语言

- ✓ 实现软件移植的一种理想的方法
- ✓ 较难实现

兼容机：由不同公司厂家生产的具有相同系统结构的计算机。

➤ 2、系列机： 由同一厂家生产的具有相同的系统结构，但具有不同组成和实现的一系列不同型号的机器。

- 较好地解决软件开发要求系统结构相对稳定与器件、硬件技术迅速发展的矛盾。
- ✓ **软件兼容**
 - **向上（下）兼容：**按某档机器编制的程序，不加修改就能运行于比它高（低）档的机器。
 - **向前（后）兼容：**按某个时期投入市场的某种型号机器编制的程序，不加修改地就能运行于在它之前（后）投入市场的机器

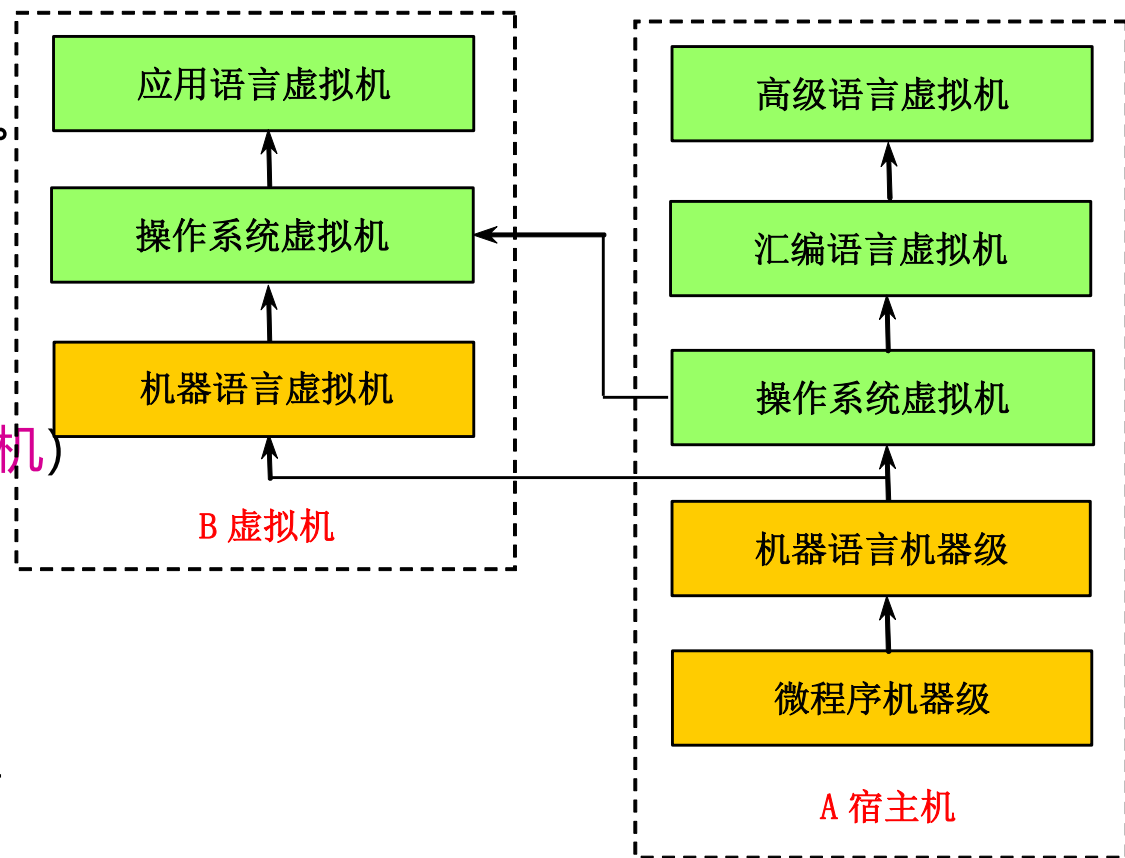




1.4.2 软件对系统结构的影响

3. 模拟和仿真

- ✓ 使软件能在具有不同系统结构的机器之间相互移植。
 - 在一种系统结构上实现另一种系统结构。
 - 从指令集的角度来看，就是要在一种机器上实现另一种机器的指令集。
- ✓ **模拟**：用软件的方法在一台现有的机器（称为**宿主机**）上实现另一台机器（称为**虚拟机**）的指令集。
 - 通常用解释的方法来实现-----**主存**。
 - 运行速度较慢，性能较差。
- ✓ **仿真**：用一台现有机器（**宿主机**）上的微程序去解释实现另一台机器（**目标机**）的指令集。
 - 运行速度比模拟方法的快（微程序级执实现另外一个机器的指令集）-----**控存**
 - 仿真只能在系统结构差距不大的机器之间使用。

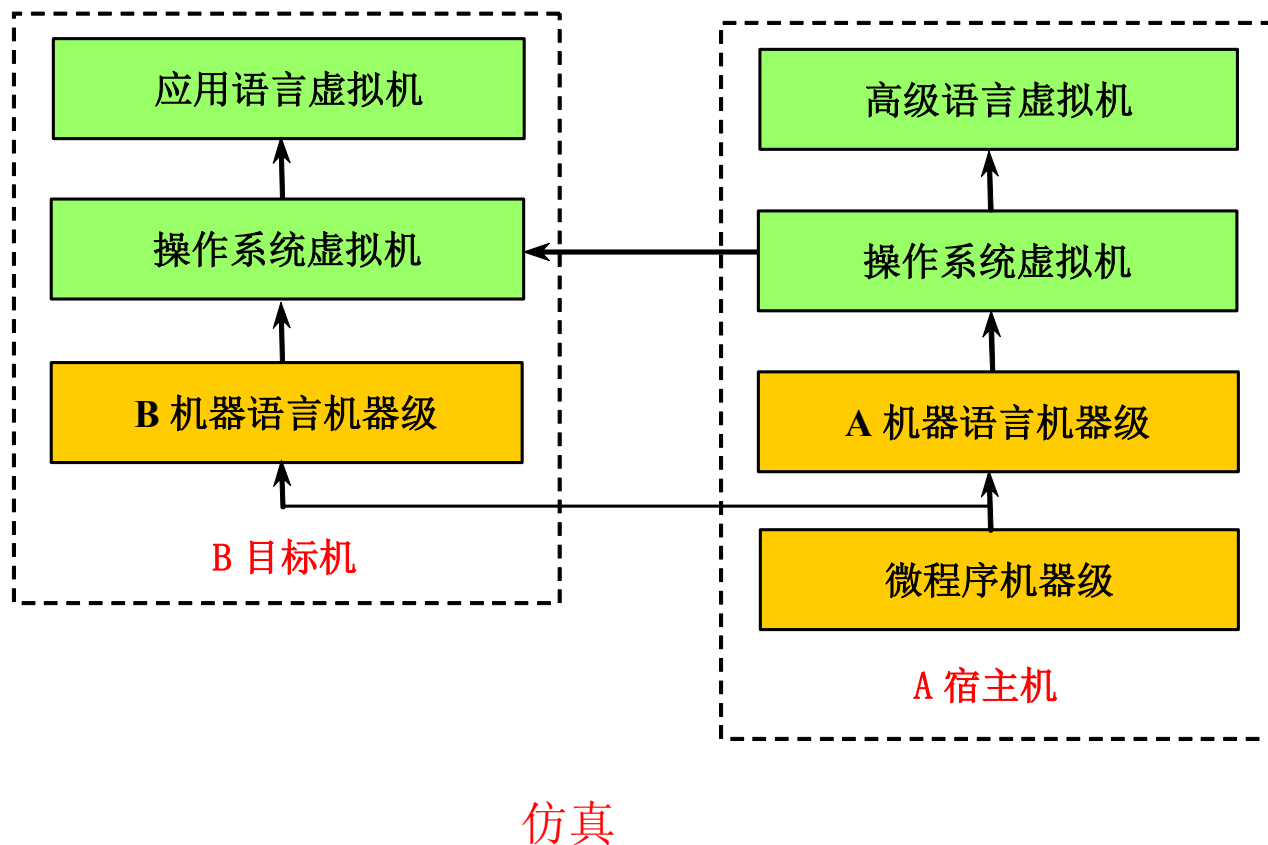


模拟

>>> 1.4.2 软件对系统结构的影响

3. 模拟和仿真

- ✓ **模拟**：用软件的方法在一台现有的机器（称为**宿主机**）上实现另一台机器（称为**虚拟机**）的指令集。
- ✓ **仿真**：用一台现有机器（**宿主机**）上的微程序去解释实现另一台机器（**目标机**）的指令集





1.4.3 器件发展对系统结构的影响

➤ 1、推动计算机系统结构不断发展的最活跃的因素

➤ 2、摩尔定律

集成电路芯片上所集成的晶体管数目每隔18个月就翻一番。

➤ 3、计算机的分代主要以器件作为划分标准。

✓ 内存每隔2年容量翻一番，价格降低1倍等，CPU性能每隔1.5年翻一番（数据资料，从1986年起）

✓ 它们在器件、系统结构和软件技术等方面都有各自的特征。

- SMP：对称式共享存储器多处理机

MPP：大规模并行处理机

MP：多处理机



1.4.3 器件发展对系统结构的影响

分代	器件特征	结构特征	软件特征	典型实例
第一代 (1945—1954年)	电子管和继电器	存储程序计算机 程序控制I/O	机器语言 汇编语言	普林斯顿ISA, ENIAC, IBM 701
第二代 (1955—1964年)	晶体管、磁芯 印刷电路	浮点数据表示 寻址技术 中断、I/O处理机	高级语言和编译 批处理监控系统	Univac LAPC, CDC 1604, IBM 7030
第三代 (1965—1974年)	SSI和MSI 多层印刷电路 微程序	流水线、Cache 先行处理 系列机	多道程序 分时操作系统	IBM 360/370, CDC 6600/7600, DEC PDP-8
第四代 (1975—1990年)	LSI和VLSI 半导体存储器	向量处理 分布式存储器	并行与分布处理	Cray-1, IBM 3090, DEC VAX 9000, Convax-1
第五代 (1991年—)	高性能微处理器高 密度电路	超标量、超流水 SMP、MP、MPP 机群	大规模、可扩展 并行与分布处理	SGI Cray T3E, IBM SP2, DEC AlphaServer 8400



1.4.4 应用对系统结构的影响

- 1、不同的应用对计算机系统结构的设计提出了不同的要求。
- 2、应用需求是促使计算机系统结构发展的最根本的动力。
- 3、一些特殊领域：需要高性能的系统结构
 - ✓ 高结构化的数值计算：气象模型、流体动力学、有限元分析
 - ✓ 非结构化的数值计算：蒙特卡洛模拟、稀疏矩阵
 - ✓ 实时多因素问题：语音识别、图象处理、计算机视觉
 - ✓ 大存储容量和输入输出密集的问题：数据库系统、事务处理系统
 - ✓ 图形学和设计问题：计算机辅助设计
 - ✓ 人工智能：面向知识的系统、推理系统等



- 1、并行性：计算机系统在同一时刻或者同一时间间隔内进行多种运算或操作。
只要在时间上相互重叠，就存在并行性。
- ✓ 同时性：两个或两个以上的事件在同一时刻发生。
 - ✓ 并发性：两个或两个以上的事件在同一时间间隔内发生。
 - ✓ 从处理数据和执行程序两个方面考虑并行性

>>> 1.5.1 并行性的概念

2. 从处理数据的角度来看，并行性等级从低到高可分为：

✓ **字串位串**：每次只对一个字的一位进行处理。

最基本的串行处理方式，不存在并行性。

✓ **字串位并**：同时对一个字的全部位进行处理，不同字之间是串行的。

开始出现并行性。

✓ **字并位串**：同时对许多字的同一位（称为**位片**）进行处理。

具有较高的并行性

✓ **全并行**：同时对许多字的全部位或部分位进行处理。

最高一级的并行。

>>> 1.5.1 并行性的概念

3. 从执行程序的角度来看，并行性等级从低到高可分为：

- ✓ **指令内部并行**：单条指令中各微操作之间的并行。（编译器或硬件实现）
- ✓ **指令级并行**：并行执行两条或两条以上的指令。（流水可解决：超标量、超流水线技术等）
- ✓ **线程级并行**：并行执行两个或两个以上的线程。
通常是以一个进程内派生的多个线程为调度单位。（利用操作系统将程序分发为多线程）
- ✓ **任务级或过程级并行**：并行执行两个或两个以上 的过程或任务（程序段）
以子程序或进程为调度单元。（例如同一台计算机处理图、数据计算、下载内容等）
- ✓ **作业或程序级并行**：并行执行两个或两个以上的 作业或程序。（例如多台机器）

>>> 1.5.2 提高并行性的技术途径（三种）

三种并行技术

➤ 时间重叠：

引入时间因素，让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部分，以加快硬件周转而赢得速度。

➤ 资源重复（多用在：多体存储器、多操作部件、阵列处理机等）

引入空间因素，以数量取胜。通过重复设置硬件资源，大幅度地提高计算机系统的性能。

➤ 资源共享

这是一种软件方法，它使多个任务按一定时间顺序轮流使用同一套硬件设备。

>>> 1.5.3 单机系统中并行性的发展

➤ 在发展高性能单处理机过程中，起主导作用的是时间重叠原理。

✓ 实现时间重叠的基础：部件功能专用化

- 把一件工作按功能分割为若干相互联系的部分；
- 把每一部分指定给专门的部件完成；
- 然后按时间重叠原理把各部分的执行过程在时间上重叠起来，使所有部件依次分工完成一组同样的工作。
- 例如取指、译码、取操作数、执行、写回等

单处理机的资源共享实质：用单处理机模拟多处理机功能，形成所谓虚拟机的概念

>>> 1.5.4 多机系统中并行性的发展 (1、2、3)

- **多机系统**：多个独立的计算机（每个都有自己的处理器、内存和操作系统）通过高速互连网络连接而成的系统。
- 通过消息传递的方式进行通信与协作，以共同完成一个大型任务。

规模：从少量节点 -> 大规模集群 -> 超大规模数据中心 -> 全球级泛在计算

架构：从同构 -> 异构；从紧耦合 -> 资源池化

目标：从追求峰值性能 -> 兼顾效率、成本、灵活性

>>> 1.5.4 多机系统中并行性的发展

- **核心**：克服通信瓶颈，高效协同一切可用的计算资源
- **展望**：并行即服务，计算力像电力一样无处不在，随需可取

>>> 1.5.4 多机系统中并行性的发展 (1、2、3)

- 1、多机系统遵循时间重叠、资源重复、资源共享原理（技术），发展为3种不同的多处理机（发展方向）：

同构型多处理机、

异构型多处理机、

分布式系统

资源重复（多同类型处理机
并行多任务）

目的：为了提高系统的性能

时间重复（不同类型处理机
，利用时间重叠）

目的：为了提高系统的效率

资源共享（不同网
络、地理分散）

目的：兼顾系统性能
和效率

>>> 1.5.4 多机系统中并行性的发展 (1、2、3)

➤ 2、耦合度： 反映多机系统中各机器之间物理连接的紧密程度和交互作用能力的强弱。

- ✓ 紧密耦合系统（直接耦合系统）：在这种系统中，计算机之间的物理连接的频带较高，一般是通过总线或高速开关互连，可以共享主存。
- ✓ 松散耦合系统（间接耦合系统）：一般是通过通道或通信线路实现计算机之间的互连，可以共享外存设备（磁盘、磁带等）。机器之间的相互作用是在文件或数据集一级上进行。

表现为两种形式：

- 多台计算机和共享的外存设备连接，不同机器之间实现功能上的分工（功能专用化），机器处理的结果以文件或数据集的形式送到共享外存设备，供其它机器继续处理。
- 计算机网，通过通信线路连接，实现更大范围的资源共享。

>>> 1.5.4 多机系统中并行性的发展 (1、2、3)

3. 功能专用化 (实现时间重叠)

- ✓ 专用外围处理机

例如：输入/输出功能的分离

- ✓ 专用处理机

如数组运算、高级语言翻译、数据库管理等，分离出来。

- ✓ 异构型多处理机系统

由多个不同类型、至少担负不同功能的处理机组成，它们按照作业要求的顺序，利用时间重叠原理，依次对它们的多个任务进行加工，各自完成规定的功能动作。

>>> 1.5.5 并行机的发展变化

并行机的发展可分为4个阶段。

➤ 1、并行机的萌芽阶段（1964年～1975年）

- ✓ 20世纪60年代初期
 - CDC6600：非对称的共享存储结构，中央处理机采用了双CPU，并连接了多个外部处理器。
- ✓ 60年代后期，**一个重要的突破**
 - 在处理器中使用流水线和重复设置功能单元，所获得的性能提高是明显的，并比单纯地提高时钟频率来提高性能更有效。
- ✓ 在1972年，Illinois大学和Burroughs公司联合研制Illiac IV SIMD计算机（64个处理单元构成的）
 - 在1975年 Illiac IV系统（16个处理单元构成）

>>> 1.5.5 并行机的发展变化

并行机的发展可分为4个阶段。

➤ 2、向量机的发展和鼎盛阶段（1976年～1990年）

- ✓ 1976年，Cray公司推出了第一台向量计算机Cray-1
- ✓ 在随后的10年中，不断地推出新的向量计算机。包括：CDC的Cyber205、Fujitsu的VP1000/VP2000、NEC的SX1/SX2以、我国的YH-1等
- ✓ 向量计算机的发展呈两大趋势
 - 提高单处理器的速度
 - 研制多处理器系统

>>> 1.5.5 并行机的发展变化

并行机的发展可分为4个阶段。

➤ 3、MPP出现和蓬勃发展阶段（1990年～1995年）

✓ 早期的MPP

- TC2000（1989年）、Touchstone Delta、Intel Paragon（1992年）、KSR1、Cray T3D（1993年）、IBM SP2（1994年）和我国的曙光-1000（1995年）等。（分布存储的MIMD计算机）

✓ MPP的高端机器

- 1996年，Intel公司的ASCI Red和1997年SGI Cray公司的T3E900（万亿次高性能并行计算机）

✓ 90年代的中期，在中、低档市场上，SMP以其更优的性能/价格比代替了MPP。

>>> 1.5.5 并行机的发展变化

4. 各种体系结构并存阶段（1995年～2000年）

- ✓ 从1995年以后，PVP（并行向量处理机）、MPP、SMP、DSM（分布式共享存储多处理机）、COW等各种体系结构进入并存发展的阶段。
- ✓ MPP系统在全世界前500强最快的计算机中的占有量继续稳固上升，其性能也得到了进一步的提高。

如：ASCI Red的理论峰值速度已达到了1Tflop/s

SX4和VPP700等的理论峰值速度也都达到了1Tflop/s

>>> 1.5.5 并行机的发展变化

5. 机群蓬勃发展阶段（2000年以后）

- ✓ **机群系统**：将一群工作站或高档微机用某种结构的互连网络互连起来，充分利用其中各计算机的资源，统一调度、协调处理，以达到很高的峰值性能，并实现高效的并行计算。
- ✓ 1997年6月才有第一台机群结构的计算机进入Top500排名
- ✓ 2003年11月，这一数字已达到208台，机群首次成为Top500排名中比例最高的结构。
- ✓ 截至2008年6月，机群已经连续10期位居榜首，其数量已经达到400，占80%。

>>> 1.5.5 并行机的发展变化

- ✓ 机群已成为当今构建高性能并行计算机系统的**最常用的结构**。
- ✓ 1993年至2008年期间，Top500中机群和MPP的数量的分布情况。

